

实验一 开发环境搭建

一、实验目的

1. 掌握 Flutter SDK 的安装与配置方法
2. 了解 Dart 语言的基本开发环境
3. 熟悉 Android Studio 开发工具的使用
4. 掌握 Flutter 项目的创建与运行流程
5. 实现跨平台应用的开发与部署

二、实验环境

- 操作系统: Windows 11
- 开发工具: Android Studio Panda 2 | 2025.3.2
- 开发框架: Flutter SDK 3.41.4
- 编程语言: Dart 3.11.1
- 验证项目: 多端温控系统

三、实验内容

3.1 Flutter SDK 安装与配置

3.1.1 下载 Flutter SDK

1. 访问 Flutter 官方下载页面: <https://docs.flutter.dev/get-started/install/windows>
2. 下载最新稳定版 Flutter SDK (版本 3.41.4)
3. 解压到指定目录 (如: D:\flutter\flutter)

3.1.2 配置环境变量

1. 右键点击"此电脑" → "属性" → "高级系统设置" → "环境变量"
2. 在"系统变量"中找到 Path, 点击"编辑"
3. 添加 Flutter SDK 的 bin 目录路径: D:\flutter\bin
4. 点击"确定"保存设置

3.1.3 验证安装

打开命令提示符，运行以下命令：

```
flutter --version
```

输出结果：

```
Flutter 3.41.4 • channel stable •  
https://github.com/flutter/flutter.git  
Framework • revision ff37bef603 (9 days ago) • 2026-03-03 16:03:22  
-0800  
Engine • hash 99578ad0355da00edb26301c874a3c250a5716f5 (revision  
e4b8dca3f1) (9 days ago) • 2026-03-03 18:24:54.000Z  
Tools • Dart 3.11.1 • DevTools 2.54.1
```

3.2 Android Studio 配置

3.2.1 安装 Flutter 插件

1. 打开 Android Studio
2. 点击 **File** → **Settings** → **Plugins**
3. 切换到 **Marketplace** 标签页
4. 搜索 **Flutter** 并安装
5. 安装完成后重启 Android Studio

3.2.2 配置 Flutter SDK 路径

1. 点击 **File** → **Settings** → **Languages & Frameworks** → **Flutter**
2. 在 **Flutter SDK path** 中输入：D:\flutter
3. 点击 **Apply** → **OK**

3.3 创建 Flutter 项目

3.3.1 创建项目

1. 打开 Android Studio

2. 点击 **File** → **New** → **New Flutter Project**
3. 选择 **Flutter** 项目类型
4. 配置项目信息：
 - 项目名称: `temp_control_system`
 - 项目位置:
`C:\Users\32236\OneDrive\Desktop\MTCS\temp_control_system`
 - 组织名称: `com.example`
5. 点击 **Finish** 创建项目

3.3.2 项目结构

```
temp_control_system/
├── lib/
│   └── main.dart          # 主应用文件
├── test/
│   └── widget_test.dart  # 测试文件
├── android/               # Android 平台配置
├── ios/                   # iOS 平台配置
├── web/                   # Web 平台配置
├── windows/               # Windows 平台配置
├── macos/                 # macOS 平台配置
├── linux/                 # Linux 平台配置
├── pubspec.yaml           # 项目配置文件
└── hello_world.dart       # Dart HelloWorld 验证文件
```

3.4 实现多端温控系统

3.4.1 应用功能设计

本实验实现了一个多端温控系统，主要功能包括：

1. 用户登录功能：
 - 用户名输入框
 - 密码输入框
 - 登录验证逻辑
2. 温度监控功能：
 - 客厅温度监控
 - 卧室温度监控

- 厨房温度监控
- 卫生间温度监控

3. 多端支持:

- Android
- iOS
- Web
- Windows
- macOS
- Linux

3.4.2 登录界面实现

登录界面采用 Material Design 3 设计风格，包含以下元素：

- 渐变背景设计
- 温度计图标
- 系统标题（中英文）
- 用户名和密码输入框
- 登录按钮
- 错误提示信息



图 1. 多端温控系统登录界面

3.4.3 温控主页实现

登录成功后进入温控主页，显示以下信息：

- 用户信息卡片（用户名、角色）
- 温度监控面板（四个房间的温度显示）
- 退出登录功能



图 2. 多端温控系统主页

3.5 项目运行与测试

3.5.1 运行项目

1. 打开 Android Studio
2. 打开 `lib/main.dart` 文件
3. 选择运行设备（模拟器或真机）
4. 点击绿色运行按钮（开始按钮）或按 `Shift + F10`

3.5.2 测试账号

- 用户名: 2023210639
- 密码: 210639

3.5.3 运行结果

项目成功运行在 Android 设备上，显示登录界面和温控主页，所有功能正常运行。

四、实验结果

4.1 环境配置成功

- Flutter SDK 3.41.4 安装成功
- Dart 3.11.1 运行正常
- Android Studio 配置完成
- Flutter 和 Dart 插件安装成功

4.2 项目创建成功

- 多端温控系统项目创建完成
- 项目结构完整，包含所有平台配置
- Dart HelloWorld 验证文件创建成功

4.3 应用功能实现

- 登录功能正常运行
- 温度监控界面显示正常
- 多端支持配置完成
- 应用界面美观，交互流畅

五、实验总结

通过本次实验，我成功完成了以下任务：

- 环境搭建：**完成了 Flutter SDK 和 Android Studio 的安装与配置，掌握了跨平台开发环境的搭建方法。
- 项目开发：**创建了一个完整的多端温控系统，实现了用户登录和温度监控功能，验证了 Flutter 跨平台开发的能力。
- 问题解决：**在实验过程中遇到了 Gradle 构建失败、网络连接异常等问题，通过配置国内镜像、清理缓存等方法成功解决。
- 技术掌握：**熟悉了 Flutter 框架的基本使用，包括 Widget 组件、状态管理、页面导航等核心概念。

本次实验为后续的跨平台应用开发奠定了坚实的基础，掌握了从环境搭建到项目部署的完整流程。

六、参考资料

1. Flutter 官方文档: <https://docs.flutter.dev/>
2. Dart 官方文档: <https://dart.dev/>
3. Android Studio 官方文档: <https://developer.android.com/studio>

实验日期: 2026 年 3 月 13 日

实验人员: 邢兆丰

学号: 2023210639